



# Greedy Yaklaşımı

## Greedy Algoritmalarının Özellikleri

- Geleceği düşünmez.
- Geriye dönüp karar değiştirmez.
- Her adımda lokal optimumu seçer.
- Çok hızlıdır.
- Lokal optimumların global optimuma götürmesi gereklidir.
- Büyük veri setlerinde etkilidir.
- Bellek kullanımı düşüktür.
- Global optimum garanti değildir.

## Greedy Algoritmaların Temel Bileşenleri

Greedy algoritmaların doğru ve sistematik bir şekilde tasarlanabilmesi için belirli yapı taşlarına ayrılması gerekir.

### 1. Aday Kümesi

Problemi çözerken kullanabileceğimiz tüm olası elemanların bulunduğu kümedir.

Aday kümesi doğru tanımlanmazsa greedy algoritma yanlış başlar.

#### Örnekler:

- Activity Selection probleminde → Her etkinlik bir adaydır.
- Knapsack probleminde → Her nesne bir adaydır.
- Coin Change probleminde → Her para birimi bir adaydır.

## 2. Seçim Fonksiyonu (Selection Function)

Her adımda hangi adayın seçileceğini belirler. Greedy algoritmanın kalbi burasıdır.

Seçim fonksiyonu genellikle şu kriterlere göre çalışır:

- En küçük
- En büyük
- En erken biten
- En yüksek oran (değer/ağırlık gibi)

### Örnekler:

- Activity Selection → En erken biten etkinliği seç
- Fractional Knapsack → En yüksek değer/ağırlık oranını seç
- Coin Change → En büyük para birimini seç

## 3. Uygunluk Fonksiyonu (Feasibility Check)

Seçilen adayın mevcut çözüme eklenip eklenemeyeceğini kontrol eder.

Yani şu soruya cevap verir: "Bu seçimi yaparsam problem kısıtlarını ihlal eder miyim?"

### Örnekler:

- Activity Selection: → Seçilen etkinlik öncekiyle çakışıyor mu?
- Knapsack: → Ağırlık kapasiteyi aşıyor mu?
- MST (Minimum Spanning Tree): → Döngü oluşturuyor mu?

## 4. Amaç Fonksiyonu (Objective Function)

Optimizasyon hedefini belirler. Genellikle 2 türdür:

1. Maksimizasyon
2. Minimizasyon

### Örnekler:

- Knapsack → toplam değeri maksimize et

- Activity Selection → seçilen etkinlik sayısını maksimize et
- MST → toplam ağırlığı minimize et

# Greedy Algoritmaların Doğrulanması

## 1. Sezgisel Gerekçelendirme

- Neden bu seçim mantıklı?
- Daha iyi bir seçenek olabilir çi?

## 2. Karşılaştırmalı Argümanlar

Optimal çözümdeki seçimleri greedy seçimlerle değiştir, eğer sonuç değişmiyorsa greedy doğrudur.

Diyelim ki:

- Optimal çözüm farklı bir seçimle başlıyor
- Greedy başka bir seçim yaptı

Şunu yaparız:

- Optimal çözümdeki ilk seçimi çıkar
- Greedy'nin seçimini koy

Eğer çözüm:

- Hâlâ geçerli
- Hâlâ optimal

ise Greedy doğru

## Activity Selection Probleminin Çözümü

- **Yanlış Sezgisel Yaklaşımlar**
  - Greedy algoritma tasarlamadan önce yanlış stratejileri görmek önemlidir.
- **Strateji 1: En erken başlayan etkinliği seç**
  - Mantık: “Erken başlarsa daha çok etkinlik sığar”
  - Problem: Uzun sürebilir → diğerlerini engeller
- **Strateji 2: En kısa süren etkinliği seç**
  - Mantık: “Kısa olan daha az yer kaplar”
  - Problem: Zaman konumu önemli → çakışma olabilir
- **Strateji 3: En az çakışanı seç**
  - Hesaplaması zor ve greedy doğasıyla uyumsuz

## Activity Selection Problemi (Etkinlik Seçimi)

- **Doğru Greedy Strateji**
  - En erken biten etkinliği seç
- **Neden?**
  - Daha erken boşluk bırakır
  - Daha fazla etkinlik için alan açar

## Activity Selection Zaman Karmaşıklığı

---

- Sıralama:  $O(n \log n)$
- Tarama:  $O(n)$
- Toplam:  $\rightarrow O(n \log n)$

## Fractional Knapsack Problemi Çözümü

- Greedy algoritmasının en güçlü ve temiz çalıştığı klasik problemlerden biridir.

### Problem Tanımı

- Elimizde:  $n$  adet nesne var
- Her nesne için:
  - Değer:  $v_i$
  - Ağırlık:  $w_i$
- Ayrıca: Çantanın kapasitesi:  $W$

### Amaç:

- Çantaya koyulan nesnelerin toplam değerini **maksimize etmek**

### Kritik Özellik:

- Bu problemde: Nesneleri **parçalayabiliriz (fractional)**

Cevap: **Birim ağırlık başına en yüksek değeri olanı**

Tanım: oran =  $\frac{v_i}{w_i}$

Sıralama:  $O(n \log n)$

Seçim:  $O(n)$

Toplam:  $\rightarrow O(n \log n)$

## Coin Change Problemi Çözümü

### Problem Tanımı

Elimizde belirli para birimleri var:  $C = \{c_1, c_2, \dots, c_n\}$

Amaç: Verilen bir miktarı **en az sayıda para kullanarak** oluşturmak

**Örnek:** Para birimleri:  $\{1, 5, 10, 25\}$ , Hedef: 30

Strateji: Her adımda **en büyük para birimini seç**

### Adım Adım:

- 25  $\rightarrow$  kalan 5
- 5  $\rightarrow$  kalan 0

Sonuç:  $\rightarrow$  **2 para (optimal)**

## Coin Change için Başarılı ve Başarısız Durumlar

### • Greedy'nin Çalıştığı Durum

- ABD para sistemi: {1, 5, 10, 25}
- Bu sistemde greedy: → **her zaman optimal çözüm verir**

### • Greedy'nin Başarısız Olduğu Durum

- **Örnek:** Para birimleri: {1, 3, 4}, Hedef: 6

### • Greedy:

- 4 → kalan 2
- 1 → kalan 1
- 1 → kalan 0
- Toplam: → **3 para**

- **Optimal:** 3 + 3 = 6

- Toplam: → **2 para**

### • Neden Greedy Burada Başarısız?

- Greedy şu varsayımı yaptı:
- “En büyük parayı seçmek en iyidir”
- Ama:
- 4 seçimi → kötü bir kalan oluşturdu (2)
- Bu da çözümü bozdu

Bir para sistemi:

- Eğer greedy her zaman optimal çözüm veriyorsa **Canonical (kanonik) coin system** denir

**Örnek:**

Canonical: {1, 5, 10, 25}

Canonical değil: {1, 3, 4}

## Minimum Spanning Tree Problemi (MST)

- **Spanning Tree Nedir?**
- Bir ağın:
  - Tüm noktalarını kapsayan
  - Bağlantılı olan
  - Gereksiz döngü içermeyen yapısına
- **Spanning Tree (Yayılma Ağacı) denir.**
- **Minimum Spanning Tree (MST)**
  - Tüm olası spanning tree'ler arasında:
  - Toplam ağırlığı (maliyeti) en küçük olan yapı
- **Neden Greedy ile ilişkili?**
  - Bu problemde şu fikir işe yarar: “Her adımda en ucuz bağlantıyı seç”
- **Sezgisel Greedy Mantık:**
  - En küçük maliyetli kenarı seç
  - Eğer problem oluşturmuyorsa ekle
  - Devam et

## Greedy Kalıpları

### 1. Sıralama Tabanlı Greedy

- **Mantık:** Elemanları belirli bir kritere göre sırala , Sırayla seç
- **Örnekler:** Activity Selection → bitiş zamanına göre, Interval Scheduling

### 2. Oran (Ratio) Tabanlı Greedy

- **Mantık:** “verimlilik” ölç → en iyiyi seç
- **Örnek:** Fractional Knapsack →  $v/w$

### 3. En Küçük / En Büyük Seçim

- **Mantık:** Her adımda en ekstrem değeri seç
- **Örnek:** Coin Change → en büyük para, Huffman → en küçük frekans

### 4. Zamanlama (Scheduling) Problemleri

- **Mantık:** Çakışmayı azalt, Zamanı verimli kullan
- **Örnek:** Activity Selection

### 5. Kaynak Dağıtımı (Resource Allocation)

- **Mantık:** Kaynakları maksimum fayda sağlayacak şekilde dağıt
- **Örnek:** Knapsack

## Greedy ve Dinamik Programlama Farkı

- **Durum 1 (Fractional - Kesirli Sırt Çantası):** Evde çuvalarla altın tozu, gümüş tozu ve un var. Açgözlü (Greedy) davranıp kilosu en pahalı olan altın tozuna saldırırsın. Çantayı tamamen altınla doldurursun, yer kalırsa gümüşe geçersin. Eşyaları bölebildiğin (gram gram alabildiğin) için "Anlık en kârlı olanı al" (Greedy) taktiği kusursuz çalışır.
- **Durum 2 (0/1 Sırt Çantası):** Evde 10 kiloluk bir radyo, 20 kiloluk bir laptop ve 30 kiloluk bir televizyon var. Eşyaları bölemezsin, yarım televizyon çalamazsın. Eğer Greedy davranıp "Kilo başına en değerli olan radyoyu alayım" dersen, çantada tuhaf boşluklar kalır ve toplamda daha az para kazanırsın. Eşyalar parçalanamıyorsa, "Hangi ikisini alırsam çantayı tam doldururum ve en çok kârı ederim?" diye tüm kombinasyonları hesaplaman gerekir. İşte bu kombinasyonları hafızada tutarak hesaplama işine **Dinamik Programlama (DP)** diyoruz.

## Hangi Durumda Hangi Paradigma Kullanılmalıdır?

## 1. En Uzun Palindromik Alt Dizi (Longest Palindromic Substring)

Palindrom, tersten de aynı okunan kelimedir (Örn: KABAK). KABAK kelimesinin palindrom olup olmadığını anlamak için baştaki ve sondaki K harflerine bakarız. Eğer aynıysa, "Acaba ortadaki 'ABA' palindrom mu?" diye sorarız. Yani büyük problemi çözerken, içindeki daha küçük bir problemin (ABA'nın) cevabını kullanırız. Problemler iç içe geçmişse ve sürekli aynı küçük parçaları tekrar hesaplıyorsak (Overlapping subproblems), çözüm her zaman DP'dir.

**Çözüm Yolu:** Dinamik Programlama

## 2. Para Üstü Verme

- *Greedy (Açgözlü) Denerse:* "Hemen elimdeki en büyük parayı vereyim" der ve 8 TL'yi verir. Geriye 3 TL kalır. 3 TL'yi ödemek için elinde sadece 1 TL'likler olduğu için 3 tane 1 TL verir. Toplamda **4 madeni para** (8, 1, 1, 1) kullanmış olur.
- *DP Denerse:* "Tüm olası kombinasyonlara bakayım" der. 5 TL ve 6 TL'yi seçer. Toplam 11 TL eder ve sadece **2 madeni para** kullanmış olur. DP, Greedy'i burada yener çünkü geçmiş seçimlerin geleceği nasıl etkileyeceğini hesaplar.

**Çözüm Yolu:** Dinamik Programlama

## 3. İş Sıralama (Job Sequencing / Activity Scheduling)

Elimizde bitiş saatleri ve getirecekleri kârlar belli olan işler var. Yapmamız gereken tek şey işleri kârına göre sıralayıp, zamanı çakışmayanları "Bunu alayım, bunu da alayım" diyerek peş peşe dizmektir. Geçmişte yaptığın bir iş, gelecekteki bir işin yapısını bozmaz. Bu yüzden anlık en iyi seçimi yapmak (Greedy) burada mükemmel ve kesin sonuç verir.

**Çözüm Yolu:** Greedy.

## 4. Matris Zincir Çarpımı (Matrix Chain Multiplication)

Tıpkı Palindrom sorusundaki gibidir. A, B, C ve D matrislerini çarpacaksın diyelim.  $(AB)(CD)$  mi daha kolay, yoksa  $A(B*C)*D$  mi? Bunu bulmak için matrisleri sürekli farklı yerlerden bölüp küçük parçaların işlem maliyetini hesaplaman gerekir. Küçük parçaların sonuçları defalarca tekrar kullanılacağı için DP'dir.

**Çözüm Yolu:** DP.

## 5. Gecikmeyi En Aza İndirme (Minimize Maximum Lateness)

Bir sürü ödevin var ve hepsinin teslim tarihi farklı. "Gecikme cezasını" en aza indirmek istiyorsun. İçgüdüsel olarak ne yaparsın? Teslim tarihi en yakın olanı en önce yaparsın. İşleri sadece teslim tarihine göre sıralamak (Greedy yaklaşımı) en ideal çözümü verir.

**Çözüm Yolu:** Greedy.